



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

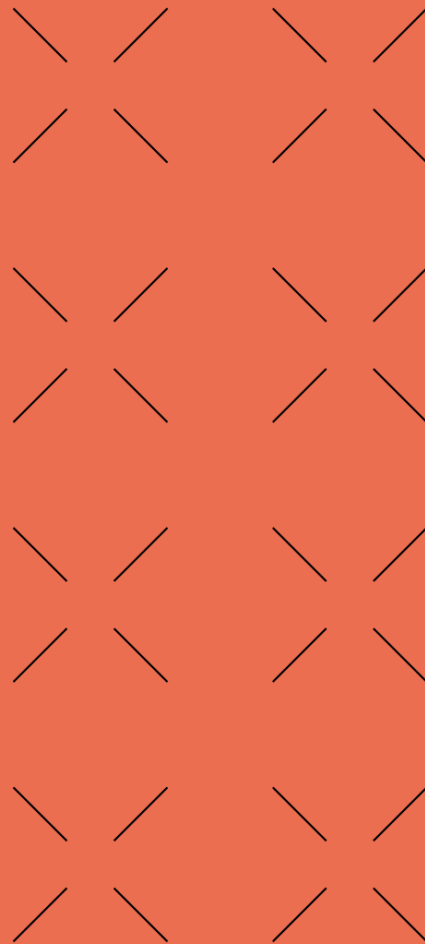
CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 05 — Funcions i mètodes

Bloc 01

1r CFGS DAW · Programació

Sergi García Barea



Què veurem en este bloc



Què és una funció?

La recepta amb nom



El primer mètode

Declarar i cridar



Paràmetres

L'entrada de la funció



return

L'eixida: tornar un valor

Tot en **Java**: el codi deixa de ser un paràgraf infinit i es converteix en un **receptari**.

Què és una funció?

La idea en una frase

“

Una funció és una **recepta amb nom**: rep dades per la porta, fa una cosa ben explicada i torna el resultat; tu només la **inviteixes pel seu nom**.



Programar copiant i enganxant és com cuinar repetint cada pas del receptari en veu alta en cada plat: funciona... **fins que tens comensals.**

El problema: el paràgraf infinit

Notas.java

Java

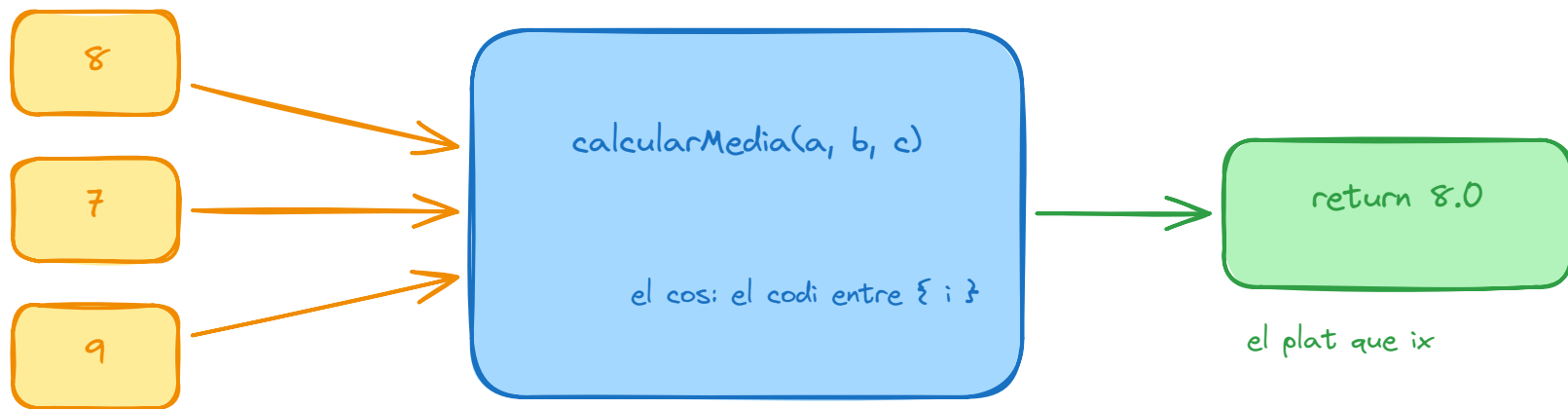
```
public class Notas {  
    public static void main(String[] args) {  
        // Calcular la mitjana de 3 notes  
        double media = (8 + 7 + 9) / 3.0;  
        System.out.println("Mitjana 1: " + media);  
        // Calcular la mitjana d'altres 3 notes... una altra volta  
        double media2 = (6 + 5 + 10) / 3.0;  
        System.out.println("Mitjana 2: " + media2);  
        // I demà, amb 30 alumnes: copiar i pegar 30 voltes  
    }  
}
```



El càlcul està escrit **dues voltes**. Si canvies la fórmula (¿i si les notes tenen pes?), has d'arreglar **totes les còpies**: una oblidada i el programa ja ment. El codi repetit no és codi: **és deute amb interessos**.

La màquina de fer plats

La funció: una recepta amb nom (entrades → procés → eixida)



els ingredients que et passen

ingredients → paràmetres

passos → cos { }

plat → return (o res: void)

La recepta amb nom

Recepta de cuina	Funció en Java
El nom («Truita de patates»)	El nom del mètode (<code>calcularMedia</code>)
Els ingredients que et passen	Els paràmetres (dades d'entrada)
Els passos dins de la recepta	El cos (el codi entre <code>{ i }</code>)
El plat que ix	El valor que torna (o res, si només fa)

● ● ● crida

Java

```
double notaMedia = calcularMedia(8, 7, 9);  
System.out.println("Mitjana 1: " + notaMedia);
```

- ✓ Dues línies on abans n'hi havia tres... i la fórmula **només està escrita una volta**. Quan canvie, canviarà en un sol lloc.

Funció o mètode? (no, no és el mateix... bé, sí)

i **Funció** és el concepte general: entrades → procés → eixida. En Python, JavaScript o C se'n diu funcions.

i **Mètode** és eixa mateixa idea **vivint dins d'una classe**, que és on viu tot en Java (fins i tot `main`, ho vas veure en la U02).

✓ En la pràctica, tothom usa els dos noms per al mateix. Si en una entrevista et pregunten la diferència: **en Java, les funcions són mètodes d'una classe.**

⚠ **Don Tip:** el nom del mètode és documentació. Un mètode anomenat `hacerCosas` és una confessió; un anomenat `calcularIVA` és un pla.

Què guanya el teu programa al tallar-lo

- 1 **No repeteixes codi.** La fórmula s'escriu una volta i s'usa 30
- 2 **Es llig millor.** `mostrarMedia(...)` explica què passa; cinc línies d'aritmètica a mitges, no
- 3 **Es prova per parts.** Comproves que `calcularMedia` funciona sense executar tot el programa
- 4 **Es repara sense por.** Arregles el tros trencat i la resta no s'assabenta

i És l'instint de la **descomposició** de la U01, però ara amb ferramenta: abans *pensaves* el problema per parts; ara **escrius** cada part per separat.

Exercici: el vident d'eixides

Vidente.java

Java

```
public class Vidente {  
    static void saludar() {  
        System.out.println("Hola, món");  
    }  
    public static void main(String[] args) {  
        saludar();  
        saludar();  
    }  
}
```



Què imprimeix? (A) Hola, món una volta · (B) dues voltes · (C) res



Solució: la B. `saludar()` es crida dues voltes i cada crida executa el seu cos. Un mètode **no s'executa sol** en declarar-se: només quan algú el crida. (La C és la trampa clàssica de «si està escrit, s'executa».)

Mini-chequeig: què és una funció?



Respon en 30 segons:

1. Quines tres coses «típiques» té una funció (taula de la recepta)?
2. S'executa el cos d'un mètode pel sol fet d'estar escrit al fitxer?
3. `main` és una funció? I `saludar()` del teu programa?
4. Menciona dos avantatges de tallar el codi en mètodes.



Respostes:

- Un **nom**, unes **entrades** (paràmetres) i un **resultat** que torna (o res)
- No: s'executa quan algú el **crida**
- Els dos són mètodes; `main` és el mètode especial per on arranca la JVM
- No repetir codi · llegibilitat · provar per parts · reparar sense trencar la resta

El teu primer mètode

La idea en una frase

“

Declarar un mètode és escriure la seua recepta (`public static void nom() { ... }`); cridar-lo és dir el seu nom amb parèntesis i punt i coma perquè **execute el seu cos**.



La **declaració** no porta punt i coma: acaba amb el seu bloc `{ ... }`. La **crida** sí: és una sentència, igual que `int x = 5;`

La primera receta

Saludo.java

Java

```
public class Saludo {  
    public static void saludar() {  
        System.out.println("Hola, classe!");  
    }  
    public static void main(String[] args) {  
        saludar();  
        saludar();  
    }  
}
```

```
$ java Saludo  
Hola, classe!  
Hola, classe!
```

i `saludar()` és la **receta** (declarada amunt);
en `main` la **cridem** dues voltes. Cada crida
executa el cos sencer.

! El nom va en **camelCase**: `calcularMedia`,
`mostrarMenu`, `esPar`. Sense espais ni accents,
la primera paraula en minúscula.

La firma, paraula per paraula

Paraula	Què significa	Quan la veuràs explicada
<code>public</code>	Visible des de qualsevol lloc	U10 (visibilitat)
<code>static</code>	Pertany a la classe, no a un objecte	U10 (mètodes static)
<code>void</code>	No torna res	Este punt 4, ara mateix
<code>saludar()</code>	El nom i els seus parèntesis	Hui

⚠ De moment escrius **sempre** `public static` davant dels teus mètodes. Sense `static`, `main` no podrà cridar-los: `non-static method cannot be referenced from a static context`.

Declarar i cridar dins de la classe

Dins de la classe: declarar a dalt, cridar a baix (o al revés)

```
public class Saludo
```

```
main(String[] args)  
saludar();  
saludar();
```

el director: la JVM el crida en arrancar

trouba

```
saludar()  
System.out.println(  
    "Hola, classe!");
```

la recepta: s'executa només quan la criden

L'ordre NO importa: Java no llig d'amunt a avall, busca per nom.
Sense static, main no podria cridar-lo (detall en la U10)

main també és un mètode

“

`main` no és especial per màgia: la JVM **el busca per eixe nom exacte** per arrancar el teu programa. Tot el que has usat des de la primera unitat era... **un mètode més**.



`public` → la JVM ha de poder veure'l · `static` → el crida **sense crear objectes** · `void` → no torna res · `String[] args` → els arguments de la línia d'ordres (U02)



Abans cridaves a un mètode sense adonar-te'n. Ara crides a tots els que et vinga de gust.

Les regles del joc

- 1 Els mètodes es declaren **dins de la classe**, mai dins d'un altre mètode (ni tan sols dins de `main`)
- 2 L'**ordre no importa**: pots cridar un mètode declarat més baix. Java no llig d'amunt a avall; **busca per nom**
- 3 Dos mètodes no poden dir-se igual **amb els mateixos paràmetres** (la sobrecàrrega arriba en la U09)



`cannot find symbol` en cridar un mètode gairebé sempre vol dir: li has posat **un nom diferent** del que té, o el crides des de fora de la classe.

Exercici: la firma trencada

A · `static public void contar() { ... }`

B · `public static contar() { ... }`

C · `public static void contar { ... }`

D · `public static void contar()`

`System.out.println("1");`

🔄 **Compila la A.** `static` i `public` són modificadors: qualsevol ordre val.

La **B** oblida el tipus de retorn (`void`).

La **C** oblida els parèntesis `()`.

La **D** oblida les claus: el cos **sempre** va entre `{ }`.

Mini-chequeig: el primer mètode



Respon sense mirar:

1. Escribe la firma completa d'un mètode `acomiar` que no torne res.
2. Quina línia (i en quin lloc) executa el cos d' `acomiar()` ?
3. Pot `main` estar declarat després d' `acomiar` ? I a l'inrevés?
4. Quina és la diferència de puntuació entre declarar i cridar?



Respostes:

- `public static void acomiar() { ... }`
- Una **crida** amb punt i coma, tipus `acomiar();` dins de `main`
- Sí als dos: l'ordre dels mètodes **no importa**
- Declarar acaba en `{ ... }` **sense** `;`; cridar és una sentència i porta `;`

Paràmetres: l'entrada

La idea en una frase

“

Els paràmetres són les variables que un mètode **rep en ser cridat**: tu passes valors en els parèntesis i el mètode treballa amb ells com si els haguera escrit ell mateix.



El nostre `saludar()` només sap dir una cosa. Per salutar a «Marta», a «Youssef» i a tota la classe... **donem-li entrades**



Una sola recepta, tres plats: el mètode ja no decideix quines dades usa. **Se les passes tu.**

Paràmetres en acció

Saludos.java

Java

```
public class Saludos {  
    public static void saludar(String nombre) {  
        System.out.println("Hola, " + nombre + "!");  
    }  
  
    public static void main(String[] args) {  
        saludar("Marta");  
        saludar("Youssef");  
        saludar("la classe sencera");  
    }  
}
```

```
$ java Saludos  
Hola, Marta!  
Hola, Youssef!  
Hola, la classe sencera!
```

i Cada paràmetre porta **tipus i nom**, com una variable: `String nombre`, `int edad`, `double nota`. Dins del cos s'usen **com variables normals**.

! Varios paràmetres se separen amb **coma** i cadascú repeteix el seu tipus: `String nombre`, `int edad`

Paràmetre ≠ argument

Moment	Nom	Exemple
Declarar el mètode	Paràmetres	<code>int a, int b</code>
Cridar el mètode	Arguments	<code>3, 4</code>

parametres.java

Java

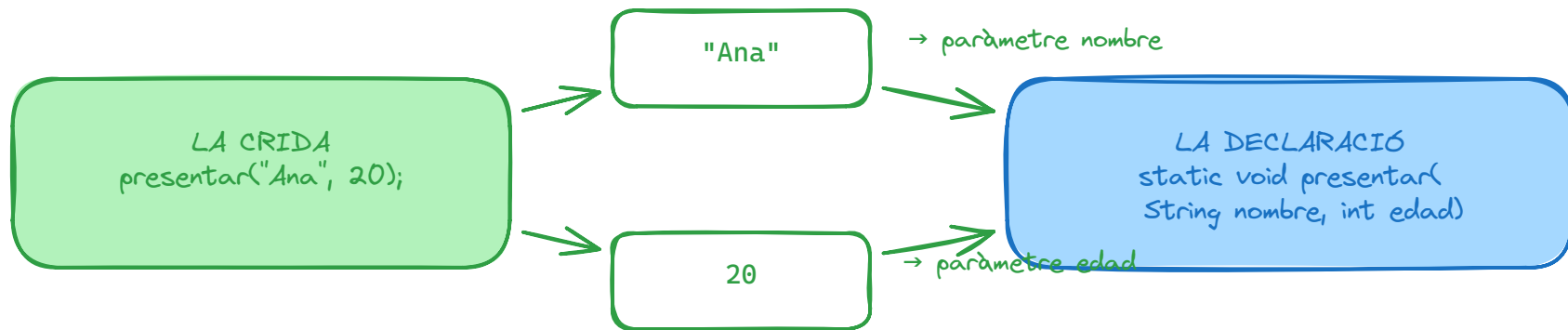
```
// paràmetres (en la declaració)
static int sumar(int a, int b) { return a + b; }
// arguments (en la crida)
int r = sumar(3, 4);
```



Nomena els paràmetres pel que **són**: `double base, double altura` s'entén; `double x, double y` obliga a llegir el cos cada volta.

La còpia que viatja

Paràmetre - argument: cada crida viatja amb la seua còpia



✓ Cada crida treballa amb una CÒPIA:
el mètode no canvia la teua variable

✗ `presentar(20, "Ana")` → incompatible types:
l'ordre dels arguments és part de la firma

el paràmetre és la cadira · l'argument és qui se senta

Regles de les entrades

- 1 **L'ordre importa:** `presentar("Ana", 20)` no és `presentar(20, "Ana")` (aquesta ni compila)
- 2 **El nombre importa:** `sumar(3)` o `sumar(3, 4, 5)` no quadren amb la firma → error de compilació
- 3 **El tipus ha d'encaixar:** un `int` cap on es demana `double` (ampliació), però un `String` on es demana `int` és un mur
- 4 Cada crida crea **les seues pròpies còpies:** `sumar(3, 4)` i `sumar(10, 20)` no es pisen

⚠ Els paràmetres **no són opcionals**: cridar `saludar();` a un mètode que espera un `String` dona
`method saludar in class ... cannot be applied to given types .`

Exercici: la crida confusa

Confusion.java

Java

```
static void pintar(String color, int veces) {  
    for (int i = 0; i < veces; i++) {  
        System.out.print(color);  
    }  
    System.out.println();  
}  
  
// en main:  
pintar("verd", 3);  
pintar(2, "blau");
```



Què passa?

- (A) verdverdverd i blaublaublau
- (B) bucle rar amb 2222...
- (C) No compila
- (D) Necessita return



Solució: la C. En `pintar(2, "blau")` arriba un `int` on ha d'anar el `String` → `incompatible types`. L'ordre dels arguments **és part de la firma**.

Mini-chequeig: paràmetres



Respon en 30 segons:

1. En `static void log(String mensaje, int nivel)`, quins són els paràmetres? I els arguments de `log("Fallo", 3)`?
2. Per què `sumar(1, 2, 3)` no compila si la firma és `sumar(int a, int b)`?
3. Es pot cridar sense parèntesis, com `saludar;`?



Respostes:

- Paràmetres: `String mensaje` i `int nivel`. Arguments: `"Fallo"` i `3`
- La firma admet **2** arguments i la crida n'envia **3**: `wrong number of arguments`
- No: sense `()` no hi ha crida. Crida = `saludar();`

| return: l'eixida

La idea en una frase

“

Un mètode amb `return` calcula i **lliura un valor** a qui l'ha cridat; un mètode `void` només executa i se calla. La firma **promet el tipus** i `return` compleix la promesa.



Fins ara els teus mètodes **feien** coses (imprimien). Ara aniran a **entregar** coses: qui crida rep el valor i **pot usar-lo**

Dos oficios, dos firmes

Suma.java

Java

```
public static int sumar(int a, int b) {  
    return a + b;  
}  
  
// en main, el valor es pot usar:  
int total = sumar(3, 4); // 7 guardat  
System.out.println(sumar(3, 4)); // 7 directe  
int doble = sumar(sumar(1, 2), 3); // 6
```

Usa return quan...

Un altre mètode
necessita el resultat

La tasca és «**calcular**»

Vols provar el valor per
separat

Usa void quan...

Només cal un **efecte**
(imprimir, guardar)

La tasca és «**fer**»

El resultat ja es veu en
pantalla



Si es diu `calcular...`, `get...` o
`es...` → segurament **torna**. Si es diu
`imprimir...` o `mostrar...` → segurament
void.

return talla l'execució

return: entrega el valor a qui crida i talla el mètode



return talla l'execució a l'instant: el primer que s'executa, acaba el mètode

```
static String clasificar(int nota) {  
    if (nota ≥ 5) return "Aprovat";  
    return "Suspès"; ← sempre s'arriba  
}
```

✗ Si un camí arriba al final
sense return →
error: missing return statement

★ Calcula → torna · Decideix → imprimeix: el `println` és ofici de qui crida

El pecat d'imprimir quan has de tornar

❌ void que imprimeix

```
static void media(int a, int b) {  
    System.out.println((a + b) /  
2.0); }
```

main **no pot guardar** la mitjana, comparar-la ni reutilitzar-la: es va imprimir i **va desaparèixer**.

✅ que torna

```
static double media(int a, int b) {  
    return (a + b) / 2.0; }
```

El valor entra al programa **com a dades de veritat**: qui crida decideix què fer amb ell.



`System.out.println` és per a `main` (o per als mètodes el ofici dels quals siga mostrar).
Qui calcula calcula; qui decideix imprimeix.

Exercici: l'endevinalla del retorn

Truco.java

Java

```
public static int truco(int x) {  
    if (x > 10) {  
        return x * 2;  
    } else if (x > 5) {  
        return x + 3;  
    }  
    return 1;  
}  
  
// què torna...  
truc(7) → ?  
truc(12) → ?
```



Tria: (A) 10 i 24 · (B) 14 i 24 · (C) 10 i 12 · (D) truco(7) no torna res



Solució: la A. `truco(7)` cau en `x > 5` i torna `7 + 3 = 10`. `truco(12)` compleix `x > 10` i torna `12 × 2 = 24`. Traça el camí línia a línia: és exactament el que fa Java.



Don Tip: `return true;` / `return false;` en mètodes `es...` o `pot...` converteix preguntes en **booleanes reutilitzables**.

Mini-chequeig: return



Respon sense mirar:

1. En `public static boolean esPar(int n)`, quina línia mínima ha de tindre el cos?
2. Quin error dona un mètode `int` amb un camí sense `return` ?
3. Es pot fer `return;` (buit) en un mètode `int` ?



Respostes:

- Almenys un `return` amb booleà: `return n % 2 == 0;`
- `missing return statement` (error de compilació)
- No: `return;` buit només val en mètodes `void`; un `int` necessita `return expressió;`

Resum del bloc 01



Funció

Recepta amb nom:
escriure una volta, usar
moltes



Firma

```
public static void  
nom() ; crida amb () i  
;
```



Paràmetres

Ordre, nombre i tipus
encaixen amb la firma



return

Talla el mètode i lliura el
valor promés



Bloc 02: l'àmbit de les variables, els 7 errors freqüents, com dividir el problema i el taller Be the Code.

